

# Sujet de Travaux Pratiques : Conception d'un Système RAG Local (Clone de NotebookLM)

## Objectifs du TP

L'objectif de cette séance est de concevoir et développer de A à Z un système **RAG (Retrieval-Augmented Generation)** entièrement local. Votre application permettra à un utilisateur de charger ses propres documents (PDF, Markdown, TXT) et d'interagir avec eux de manière intelligente, tout en garantissant la confidentialité des données (aucun appel à des API externes comme OpenAI ou autres).

Pour bien comprendre la mécanique interne d'un RAG, votre application devra proposer **deux modes de fonctionnement distincts**, activables via un bouton "Toggle" dans l'interface :

1. Un mode **"Recherche Sémantique pure"** (sans génération LLM).
2. Un mode **"Assistant RAG complet"** (avec génération LLM contrainte par les documents).

## Travail à Réaliser

### Étape 1 : Initialisation de l'Environnement et Squelette de l'Interface

1. Installez les dépendances (langchain, langchain-community, chromadb, sentence-transformers, streamlit, pymupdf).
2. Assurez-vous que le service Ollama tourne en tâche de fond sur votre machine avec un modèle chargé (ex: ollama run mistral ou qwen2.5-coder).
3. Développez le squelette de l'interface graphique (à faire avec StreamLit). Elle devra contenir :
  - Une **barre latérale** contenant une zone de téléchargement de fichiers, un bouton d'indexation et un bouton d'activation/désactivation du LLM.
  - Une **zone principale** présentant une interface conversationnelle avec un historique.

*Indication technique (Streamlit) : Regardez du côté de `st.sidebar`, `st.file_uploader`, `st.toggle()`, `st.chat_input()` et `st.chat_message()`.*

### Étape 2 : Le Pipeline d'Ingestion (Traitement des Données)

Lorsque l'utilisateur charge des fichiers et clique sur le bouton d'indexation, votre backend doit traiter ces documents en direct.

1. **Extraction** : Implémentez la lecture et l'extraction de texte pour les PDF et les fichiers textes uploadés. *Indication technique (LangChain) : Utilisez les DocumentLoaders adaptés, comme PyMuPDFLoader ou TextLoader.*
2. **Chunking (Découpage)** : Découpez le texte extrait en segments (*chunks*). Vous devrez justifier la stratégie choisie (taille des chunks, taille du chevauchement ou *overlap*).

3. **Vectorisation** : Transformez ces segments en vecteurs (Embeddings) et stockez-les dans la base. Assurez-vous que les métadonnées (nom du fichier) sont conservées. *Indication technique : Utilisez `HuggingFaceEmbeddings`*

## Étape 3 : Implémentation du Mode "Recherche Sémantique" (Toggle Désactivé)

Ce mode sert à auditer et valider le bon fonctionnement de votre base vectorielle.

1. Lorsque l'utilisateur pose une question et que l'interrupteur LLM est **désactivé**, votre système ne doit faire appel à aucun modèle génératif.
2. Interrogez la base vectorielle pour récupérer les fragments de textes dont le sens est le plus proche de la requête.
3. **Affichage** : L'interface doit afficher les extraits bruts retournés par la recherche (le contenu du chunk) ainsi que le nom du fichier source exact contenu dans les métadonnées de l'objet document.

## Étape 4 : Implémentation du Mode "RAG Complet" (Toggle Activé)

C'est l'aboutissement de l'application. Si le Toggle est **activé**, l'application passe en mode "Clone NotebookLM".

1. Récupérez les fragments pertinents (comme à l'Étape 3) pour les utiliser comme contexte.
2. **Ingénierie de Prompt** : Construisez un prompt système strict ordonnant au LLM de répondre *exclusivement* en se basant sur le contexte fourni. *Indication technique : Utilisez `PromptTemplate` pour injecter dynamiquement le {context} et la {question} dans votre texte d'instruction.*
3. Envoyez la requête finale au modèle local et affichez la réponse.
4. **Transparence** : Sous la réponse du LLM, ajoutez un élément visuel permettant à l'utilisateur de déplier et vérifier les extraits de texte ayant servi de contexte.

## Modalités de Rendu

À l'issue de la séance (ou du délai imparti), vous devrez fournir :

1. **Le code source complet** de votre application (app.py), proprement commenté.
2. **Date limite**: le prochain regroupement
3. **Des aléas pendant la séance**